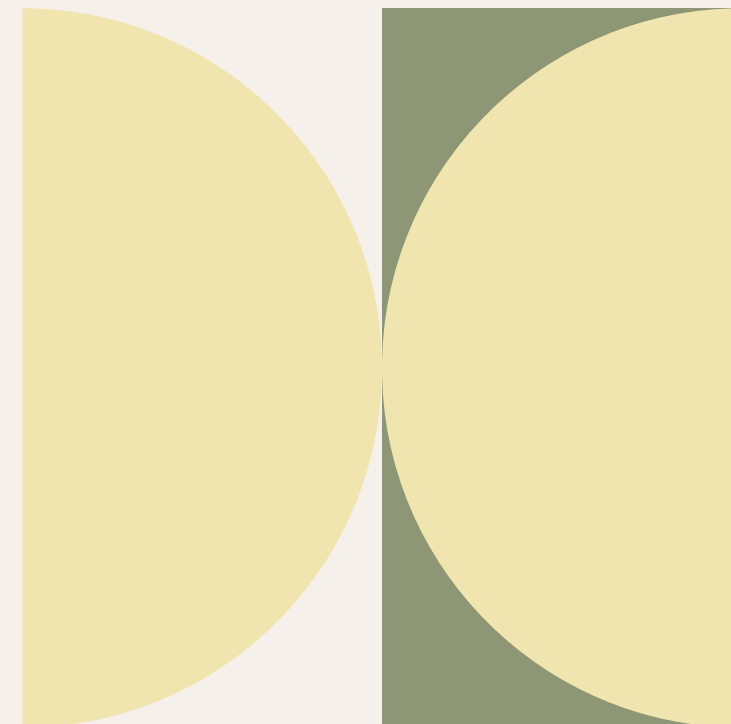
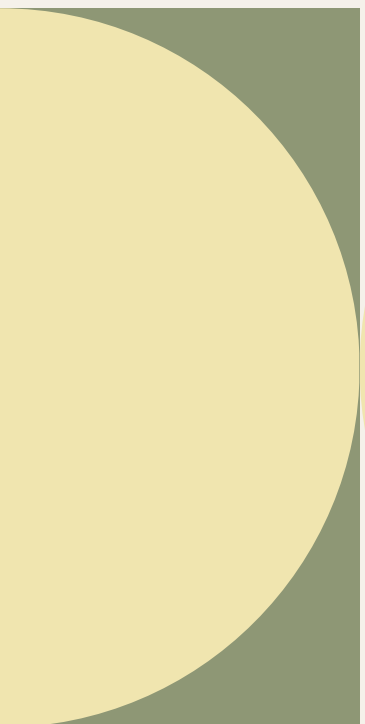

ALAN LAM PÁEZ

1 de octubre de 2026

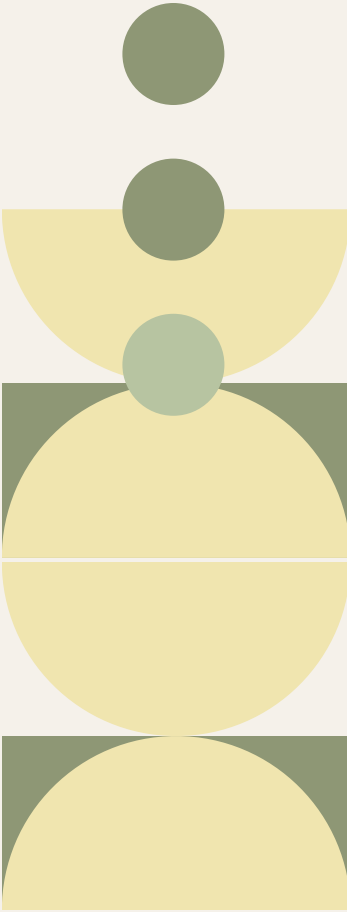


QA Y SOFTWARE TESTING

PRESENTACIÓN FINAL

¿QUÉ ES QA Y SOFTWARE TESTING?

- **Quality Assurance (QA):** Es un conjunto de actividades y procesos orientados a prevenir defectos. Se enfoca en mejorar el proceso de desarrollo para que los errores no se introduzcan desde el principio.
- **Software Testing:** Es un proceso orientado a detectar defectos en el producto ya construido. Se enfoca en encontrar problemas y verificar que el software cumpla con lo esperado.

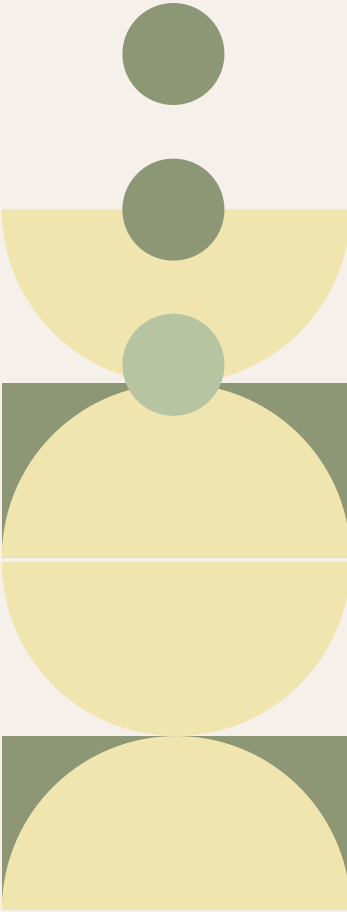


¿QUÉ ES QA Y SOFTWARE TESTING?

¿Cuál es la diferencia?

- QA es proactivo, previene el error en el proceso.
- Testing es reactivo, encuentra el error en el producto.

Objetivo principal (de ambos): Entregar software de calidad que cumpla con los requisitos y satisfaga al usuario, reduciendo el riesgo de fallos en producción.



STLC: CICLO DE VIDA DE PRUEBAS

Requirements: Analizar los requisitos para identificar qué es testeable.

Test Planning: Definir la estrategia, el alcance, los recursos y el cronograma.

Test Analysis: Determinar las condiciones de prueba y los datos necesarios.

Test Design: Crear los casos de prueba detallados con sus pasos y resultados esperados.

Test Implementation: Preparar el entorno de pruebas y los scripts necesarios.

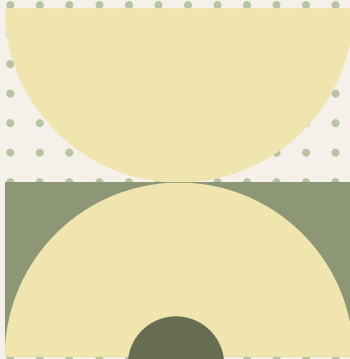
Test Execution: Ejecutar los casos de prueba y comparar los resultados.

Defect Reporting: Documentar y reportar los fallos encontrados.

Regression: Asegurar que los cambios no rompieron funcionalidades existentes.

Retesting: Verificar que los defectos corregidos funcionan.

Test Closure: Documentar los resultados, métricas y lecciones aprendidas.



TÉCNICAS DE DISEÑO DE PRUEBAS

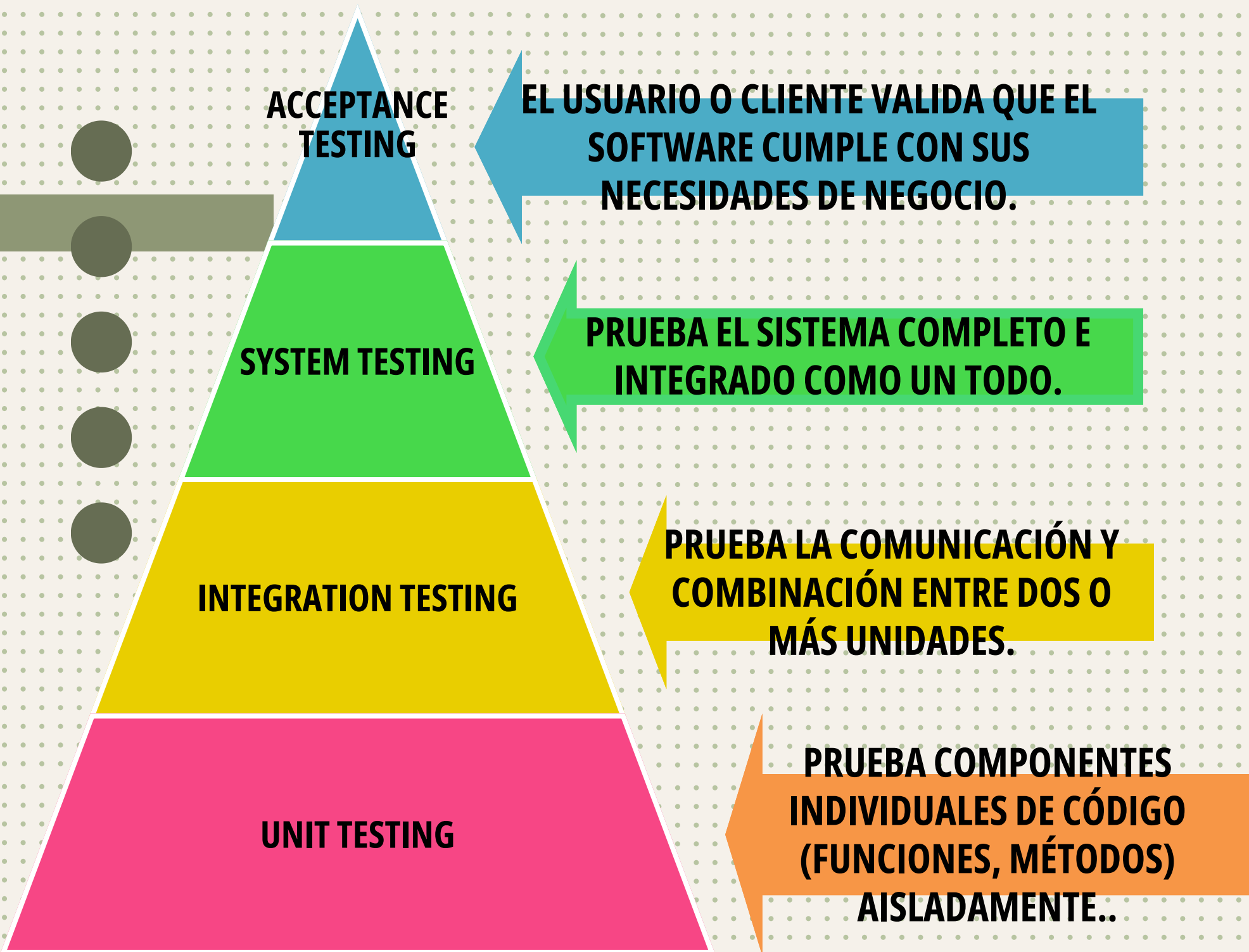
Técnica	Definición Breve	Ejemplo Visual
Partición de Equivalencias	Divide los datos de entrada en grupos que deberían comportarse igual. Se prueba un representante de cada grupo.	PARTICIONES <pre>graph LR; A([Campo "Edad" (0-120)]) --> B[Inválido si es <0, o >120]; A --> C[Niño (0-12)]; A --> D[Adulto (13-64)]; A --> E[Mayor (65-120)]; B --> F[Se prueban los valores: -1, 6, 30, 70, 121.]; C --> F; D --> F; E --> F;</pre>
Análisis de Valores Frontera	Se prueban los valores justo en los límites de cada partición, ya que allí se concentran los errores.	Partición 1 (0-12) Partición 2 (13-64) Partición 3 (65-120) LÍMITE (blue dot) FUERA DE RANGO (red dot) Valores a probar: -1, 0, 12, 13, 64, 65, 120, 121

TÉCNICAS DE DISEÑO DE PRUEBAS

Técnica	Definición Breve	Ejemplo Visual
Tabla de Decisión	Útil para lógica de negocio compleja. Se listan todas las combinaciones de condiciones y las acciones resultantes.	<pre>graph TD; D1{¿Es Premium? & Compra >100} -- SI --> A1[Aplicar 20% desc.]; D1 -- NO --> D2{¿Es Premium? & Compra ≤ 100}; D2 -- SI --> A2[Aplicar 10% desc.]; D2 -- etc... --> Etc[etc...];</pre>
Transición de Estados	Modela el comportamiento del sistema como una máquina de estados. Se prueban las transiciones válidas e inválidas.	<pre>graph LR; A[Acceder a panel de admin desde URL] --> D{¿Logueado?}; D -- SI --> B[Ingresar a panel]; D -- NO --> C[Lanzar a página de Login];</pre>

NIVELES DE PRUEBAS

TIPOS DE PRUEBAS



Functional Testing	Verifica que las funciones del software operen según lo especificado.
Regression Testing	Asegura que los cambios recientes no hayan roto funcionalidades existentes.
Smoke Testing	Prueba rápida y superficial para confirmar que la build es estable para seguir probando.
Sanity Testing	Prueba enfocada en verificar una corrección específica o una nueva funcionalidad.
Exploratory Testing	Prueba simultánea de aprendizaje y diseño, donde el tester explora el software sin un guion predefinido .

QA TESTER ACTUALMENTE

¿QUÉ HABILIDADES NECESITA ACTUALMENTE UN QA TESTER?

Hard Skills

- Fundamentos de testing manual: Diseño de casos, exploratorio, reporte de bugs.
- Programación/Scripting: Python, JavaScript o Java (nivel funcional para escribir/leer tests).
- API Testing: Entender REST/GraphQL, usar herramientas como Postman.
- SQL Básico: Consultar bases de datos para validar datos e investigar bugs.
- CI/CD: Entender pipelines (GitHub Actions, Jenkins) y cómo se integran las pruebas.
- Automatización: Conocer frameworks (Selenium, Cypress, Playwright) y saber cuándo usarlos.

Herramientas Esenciales

- Gestión de Pruebas: Jira, TestRail, Zephyr.
- Automatización: Selenium, Cypress, Playwright.
- API: Postman, SoapUI.
- CI/CD: Git, Jenkins, GitHub Actions, Docker.
- Version Control: Git.

Soft Skills

- Pensamiento Analítico: Descomponer sistemas complejos, identificar casos límite.
- Atención al Detalle: Encontrar el bug que otros pasan por alto.
- Comunicación: Escribir reportes de bugs claros y explicar problemas técnicos a no-técnicos.
- Colaboración: Trabajar de cerca con desarrolladores y producto.